

GitHub Copilot app — the standalone desktop client's own config surface

Written 2026-09-22, split out of `copilot-desktop-apps-coverage-gap.html` 's four-surfaces disambiguation into its own doc, the same way VS Code's built-in Copilot Chat extension and Microsoft 365 Copilot already were. This is the third of those four surfaces: **the GitHub Copilot app** (`github.com/features/ai/github-app`) — a brand-new, separate desktop application, described on GitHub's own page as *"the only desktop experience for agent-driven development built natively on GitHub."* Not the classic GitHub Desktop git client, not the `copilot` CLI, and not VS Code's Copilot Chat extension — a fourth, distinct product that happens to share the "Copilot" name and, confusingly, the same `~/.copilot` home-directory name the CLI adapter already targets.

Where this stands: never targeted by this codebase's existing adapter. Bringing it into scope is possible in principle — a real, per-device configuration surface exists — but every path to using it carries a distinct kind of risk from anything else this investigation has covered: not a network-protocol mismatch, but a live, undocumented, actively-written SQLite database sitting behind an app that isn't open source.

What was actually found (live install, 2026-09-15)

`/Applications/GitHub Copilot.app`, home directory `~/.copilot/` — sharing a name with the CLI's own home directory, which is itself a little misleading, since the two products don't share files.

The files this codebase's existing adapter writes to don't exist on this real, actively-used install at all:

```
~/.copilot/
├─ config.json           # app metadata only – NOT settings.json
├─ data.db (+ -shm, -wal) # SQLite, actively open – real config lives here
├─ session-store.db
├─ repo-metadata-cache.db
├─ servers/              # empty
├─ ide/                   # empty
├─ installed-plugins/     # empty
└─ logs/, run/, session-state/, hooks/, media-cache/
```

No `mcp-config.json`! No `settings.json`. `config.json` only carries UI/app metadata (`appTipShown`, `reasoningSummariesCleanupDone`, ...) — nothing resembling an MCP registry or an LLM redirect.

Where the real config actually lives: `data.db` (SQLite)

Two tables matter for the LLM leg:

```
CREATE TABLE model_providers (  
  id TEXT PRIMARY KEY NOT NULL,  
  name TEXT NOT NULL,  
  type TEXT NOT NULL DEFAULT 'openai',  
  settings_json TEXT NOT NULL DEFAULT '{}',  
  account_id TEXT REFERENCES accounts(id) ON DELETE CASCADE,  
  -- + created_at, updated_at  
);  
  
CREATE TABLE provider_models (  
  id TEXT PRIMARY KEY NOT NULL,  
  provider_id TEXT NOT NULL REFERENCES model_providers(id) ON DELETE CASCADE,  
  model_id TEXT NOT NULL,  
  wire_model TEXT,  
  -- + display_name, max_prompt_tokens, max_output_tokens, wire_api_override, ...  
);
```

The live row, on the install this was checked against:

```
model_providers:  
  id          = github_copilot:eb941f95-68df-4c6a-873d-5a3f4c959166  
  name        = GitHub Copilot  
  type        = github_copilot  
  settings_json = {"authKind":"none","baseUrl":"","  
                  "headersJson":"{}","wireApi":"responses"}  
  account_id   = eb941f95-68df-4c6a-873d-5a3f4c959166
```

`baseUrl` is the field. It's a JSON key, inside a TEXT column, inside a SQLite row — not a file this codebase's `apply_config_delta` /JSON-merge machinery can touch. `provider_models` was empty (0 rows) on the install checked.

No MCP-server-definition table exists anywhere in the schema. The only MCP-adjacent tables are session-scoped event logs (`session_managed_mcp_event_display_names`, `session_mcp_event_sources`) — tracking, not configuration. This lines up with an unverified public search finding that MCP config for GitHub's newer surfaces may have moved to a per-project file (`<project>/copilot/mcp-config.json`) rather than a per-user-home file — not confirmed against this exact app; worktree checkouts inspected during this investigation had no such file, but those may not be representative.

`model_providers` is the BYOK feature, not an obscure internal field

Re-verified directly against the app's compiled binary (`strings` on `/Applications/GitHub Copilot.app/Contents/MacOS/github`, v1.1.21 — not guessed), following up on a GitHub changelog claiming the app supports BYOK ("Settings → Model Providers", 2026-06-23).

⚠ **Evidence class, stated plainly:** everything below came from `strings` on a compiled binary, not from a formally issued GitHub document or public source code — this app isn't open source. The embedded SQL text is real (it's the literal shipped code, not a paraphrase), so the *schema itself* can be trusted as accurate. But nothing here is GitHub-published or GitHub-supported: no commitment it stays stable across an update.

- **The binary's embedded SQL migration history is readable, and it names the design.** A code comment reads: *"BYOK (Bring Your Own Key) tables. Stores user-configured model providers and the models they expose to the app... See plan in docs/byok.md... for the full design."* This is a real, first-party, named feature — not a coincidental field.
- `model_providers` is literally the same table as `byok_providers`. A later migration in the same embedded history runs `ALTER TABLE byok_providers RENAME TO model_providers`. The `baseUrl` field found above is the BYOK storage, post-rename. **The full pre-rename schema is recoverable from the binary:**

```
CREATE TABLE byok_providers (  
  id TEXT PRIMARY KEY NOT NULL,  
  name TEXT NOT NULL,  
  type TEXT NOT NULL CHECK (type IN  
    ('openai','azure','anthropic','ollama','foundry_local','custom')),  
  base_url TEXT NOT NULL,  
  wire_api TEXT NOT NULL DEFAULT 'completions'  
    CHECK (wire_api IN ('completions','responses')),  
  auth_kind TEXT NOT NULL DEFAULT 'api_key'  
    CHECK (auth_kind IN ('api_key','bearer_token','none')),  
  headers_json TEXT NOT NULL DEFAULT '{}',  
  -- + created_at, updated_at  
);  
CREATE TABLE byok_models (  
  id TEXT PRIMARY KEY NOT NULL,  
  provider_id TEXT NOT NULL REFERENCES byok_providers(id) ON DELETE CASCADE,  
  model_id TEXT NOT NULL,  
  -- + wire_model, display_name, max_prompt_tokens, max_output_tokens, ...  
);
```

`type='custom'` with `auth_kind='none'` is exactly the shape a local, no-auth endpoint needs. A session then selects it via a `sessions.model = 'byok:<provider_id>:<model_id>'` string, confirmed from the same embedded SQL. Secrets are kept out of the DB entirely, in the OS keychain under `byok:<provider_id>:apiKey` / `...:bearerToken`.

A real gap in the evidence, worth stating plainly rather than smoothing over

An earlier pass of this investigation treated the URL-override behavior as "*confirmed at the runtime code level, not just the schema*" — but the source actually read for that confirmation was `openAIProvider.ts` / `anthropicProvider.ts` inside `microsoft/vscode`'s bundled Copilot extension: **a different product**, VS Code's own separate BYOK feature, not this standalone app's code. That source does show a real, non-hardcoded `model.url` driving the actual completion request rather than a fixed default — a genuine, useful finding — but it's evidence about VS Code's BYOK implementation, not this app's. The two products share a name for the feature and a broadly similar schema shape, which is almost certainly why they got conflated; it doesn't mean they share code. **For the standalone GitHub Copilot app specifically, only the schema is confirmed** (the binary's embedded SQL, read directly) — whether a `type='custom'` provider's real inference call actually uses the stored `base_url` rather than some hardcoded default remains schema-plausible, not runtime-confirmed, for this app. Closing that gap would need reading this app's own decompiled request-building logic, or a live test with a real `custom` provider row and a listener to observe whether it's actually hit.

Two distinct BYOK mechanisms exist — only one is reachable from a local proxy

GitHub's enterprise admin console has its own, separate "Configure custom models" flow (AI controls → Copilot), confirmed **entirely server-side**: the admin's key and endpoint are held by GitHub's platform and pushed down to clients through it. That one is useless for a local interception approach — GitHub's servers can't reach a `127.0.0.1` loopback listener. The relevant one is the **per-device** `byok_providers` table above, confirmed to work "without a signed-in GitHub account" and be "global per app install" — evaluated locally, by the app itself.

No file, CLI, or programmatic path is documented for the per-device version — GUI-only (`Settings` → `Model providers` → `Add provider`). Automating it without user interaction means either a direct DB write or driving the actual GUI once per device.

The enterprise flow's actual parameters, and where inference calls really go

Enumerated from GitHub's admin how-to doc (`enable-custom-models`), every field the flow exposes:

Provider	Supported	URL configurable?
OpenAI, Anthropic, xAI	Yes — key entered, then a "fetch models" button queries the provider server-side	No URL field described — implies a fixed, known endpoint per provider
AWS Bedrock, Google AI Studio	Listed as supported	Not detailed in what this doc surfaced

Provider	Supported	URL configurable?
Microsoft Foundry	Yes	Yes — explicit "Deployment URL" field, manually entered, plus manual per-model "Model ID" entries (no auto-fetch)
OpenAI-compatible providers	Listed as supported	Genuine documentation gap — this category needs <i>some</i> base-URL field by definition, but the fetched page doesn't describe one anywhere

Other fields: **Name** (shown in the model picker), **API Key** (with an explicit least-privilege-scoping recommendation), and **Access** scoping ("Allow for all organizations" or per-organization).

Whether actual inference calls get relayed through GitHub's own backend is architecture-inferred, not directly confirmed by any doc — held to a lower confidence bar than the schema findings above:

- **Enterprise custom models:** very likely relayed through GitHub's CAPI backend for every real inference call, not just the one-time model-list fetch. Basis: the provider API key is entered once, centrally, and never described as being handed to client machines — a client that never holds the credential can't call the provider directly, so GitHub's own backend is almost certainly the one making that outbound call and relaying the response. Reasoned from the security model, not a quoted statement that calls are proxied.
- **Personal/per-device BYOK** (the `byok_providers` table): the opposite architecture — the key lives in the local OS keychain, and the flow is documented to work "*without a signed-in GitHub account.*" No reason for GitHub's servers to be in the loop; the client almost certainly calls the configured provider (or a local `type='custom'` endpoint) **directly**.
- Either way, this doesn't change the operative conclusion: the enterprise flow is unreachable from a local proxy because the calling machine is GitHub's servers, not the endpoint; the per-device flow is reachable in principle precisely because the call originates locally — modulo the runtime-confirmation gap noted above.
- **A real dependency for the per-device path:** the two BYOK tiers are additive by default (enterprise-pushed models and personal BYOK entries coexist in the same model picker), but there's a one-directional admin kill-switch specifically over the personal side. If an org has disabled it, personal BYOK is unavailable **org-wide**, before any DB-write question even applies — worth checking that policy's state as an early step in any rollout.

Why a direct database write still isn't a "wrong path" fix

- **The DB is open and actively written to right now** — the app was running with the WAL file growing during this investigation. Writing into it from outside the app risks corruption, or being silently overwritten by the app's own next commit, unless done with real SQLite transaction

discipline (and ideally the app's own cooperation — e.g. writing only while confirmed not mid-write, which nothing in this codebase's current tooling attempts).

- **No existing code path fits.** Every adapter in this repo assumes a mergeable JSON/TOML *file* (`ConfigDelta` → `config_render` → `apply_config_delta` 's file merge). None of that machinery knows how to address a database row.
- **The schema is confirmed from the shipped binary itself, not guessed — but that isn't the same as GitHub-published or GitHub-supported.** This came from `strings` on a private binary, not a formal doc or open-source repo, so nothing commits GitHub to this shape staying the same release to release, and writing outside the app's own code still bypasses whatever validation/side effects the real "Add provider" flow performs.

Recommended paths forward

1. **Don't attempt a live-DB write without explicit product sign-off**, admin privilege notwithstanding. Root/LocalSystem removes the permissions barrier, not the concurrency hazard — the write still races the app's own live process, can be silently overwritten by its next flush, and skips whatever validation/keychain/cache-invalidation the real "Add provider" flow performs. The blast radius is the app's entire local state (sessions, workspaces, worktrees), not a small, single-purpose config file.
2. **The schema risk is real and confirmed, which changes the shape of "new product design work," not its necessity.** Two realistic directions: - Build careful, transaction-safe SQLite write tooling against this now-confirmed schema, with the same "narrow trust, confirm against live install, never guess a schema" discipline established elsewhere in this codebase — and still accept re-validation on every app update, since GitHub doesn't publish this schema. - Push for a supported, non-GUI way to provision the per-device BYOK provider — GitHub's own docs confirm none exists today.
3. **Before either direction, close the runtime-confirmation gap.** Confirm — against this app's own code or a live test, not VS Code's — that a `type='custom'` provider's real inference call actually reads `base_url` at request time. Everything above this depends on that being true for this app specifically.

Open questions

- Is the GitHub Copilot app actually a stated priority for LLM-traffic brokering, given the live-DB risk profile above? If not, this doc is the record of why it's out of scope, not a to-do.
- Does GitHub expose (or plan to expose) any supported external configuration surface for this app? Worth checking their own repo/issue tracker for the app (separate from `github/copilot-cli`) if one exists publicly.

- Does a `type='custom'` provider's inference call actually use the stored `base_url` for *this* app, not just VS Code's separate implementation of the same idea? Unresolved — see the evidence-gap section above.